



NATIONAL SENIOR CERTIFICATE EXAMINATION
NOVEMBER 2009

INFORMATION TECHNOLOGY: PAPER II
MARKING GUIDELINES

Time: 3 hours

120 marks

These marking guidelines were used as the basis for the official IEB marking session. They were prepared for use by examiners and sub-examiners, all of whom were required to attend a rigorous standardisation meeting to ensure that the guidelines were consistently and fairly interpreted and applied in the marking of candidates' scripts.

At standardisation meetings, decisions are taken regarding the allocation of marks in the interests of fairness to all candidates in the context of an entirely summative assessment.

The IEB will not enter into any discussions or correspondence about any marking guidelines. It is acknowledged that there may be different views about some matters of emphasis or detail in the guidelines, and different interpretations of the application thereof. Hence, the specific mark allocations have been omitted.

Please note that learners who provided alternate correct responses to those given in the marking guidelines will have been given full credit.

Model Answer: Question 1 (JAVA)

File: FindAPet.java

```
import java.sql.*;
import javax.swing.*;

public class FindAPet
{
    private Connection conn;

    public FindAPet ()
    {
        try
        {
            Class.forName ("sun.jdbc.odbc.JdbcOdbcDriver");
            conn = DriverManager.getConnection ("jdbc:odbc:DRIVER={Microsoft
                Access Driver (*.mdb)};DBQ=pets.mdb");
            System.out.println ("Connection successful");
        }
        catch (Exception e)
        {
            System.out.println ("ERROR: " + e);
            e.printStackTrace ();
        }

        char choice;
        do
        {
            choice = getMenuChoice ();

            if (choice == 'A')
            {
                try
                {
                    Statement s = conn.createStatement ();
                    String query = "SELECT * FROM tblOwners ORDER BY LastName,
                        FirstName;";
                    s.execute (query);
                    ResultSet rs = s.getResultSet ();

                    while (rs.next ())
                    {
                        System.out.println (rs.getString ("FirstName") + " " +
                            rs.getString ("LastName"));
                    }
                }
                catch (SQLException e)
                {
                    System.out.println ("ERROR: " + e);
                    e.printStackTrace ();
                }
            }

            else if (choice == 'B')
            {
                try
                {
                    String city = JOptionPane.showInputDialog ("Please enter a
                        city");
                    Statement s = conn.createStatement ();
```

```

        String query = "SELECT * FROM tblOwners WHERE city = '" +
                        city + "' ORDER BY LastName, FirstName;";
        s.execute (query);
        ResultSet rs = s.getResultSet ();
        System.out.println ("\n" + city.toUpperCase ());
        while (rs.next ())
        {
            System.out.println (rs.getString ("FirstName") + " " +
                                rs.getString ("LastName"));
        }
    }
    catch (SQLException e)
    {
        System.out.println ("ERROR: " + e);
        e.printStackTrace ();
    }
}

else if (choice == 'C')
{
    try
    {
        Statement s = conn.createStatement ();
        String query = "SELECT PetType, COUNT(*) as num FROM
                        tblPets GROUP BY PetType";
        s.execute (query);
        ResultSet rs = s.getResultSet ();

        while (rs.next ())
        {
            System.out.println (rs.getString ("PetType") + ":\t" +
                                rs.getString ("num"));
        }
    }
    catch (SQLException e)
    {
        System.out.println ("ERROR: " + e);
        e.printStackTrace ();
    }
}

else if (choice == 'D')
{
    try
    {
        String type = JOptionPane.showInputDialog ("Please enter a
                                                    pet type");
        Statement s = conn.createStatement ();
        String query = "SELECT DISTINCT tblOwners.* FROM tblOwners
                        INNER JOIN tblPets ON tblOwners.ownerID =
                        tblPets.OwnerID WHERE PetType = '" + type +
                        "' ORDER BY lastname, firstname";
        s.execute (query);
        ResultSet rs = s.getResultSet ();

        while (rs.next ())
        {
            System.out.println (rs.getString ("FirstName") + " " +
                                rs.getString ("LastName"));
        }
    }
    catch (SQLException e)
    {
        System.out.println ("ERROR: " + e);
        e.printStackTrace ();
    }
}
}

```

```

else if (choice == 'E')
{
    try
    {
        String name = JOptionPane.showInputDialog ("Please enter
                                                    the pet's name:");
        String type = JOptionPane.showInputDialog ("Please enter
                                                    the pet's type:");
        String breed = JOptionPane.showInputDialog ("Please enter
                                                    the pet's breed:");
        String owner = JOptionPane.showInputDialog ("Please enter
                                                    the owner's last name:");
        String actualOwner = "";

        Statement s = conn.createStatement ();
        String query = "SELECT * FROM tblOwners WHERE lastname = '"
                        + owner + "'";
        s.execute (query);
        ResultSet rs = s.getResultSet ();
        int countowners = 0;
        while (rs.next ())
        {
            countowners ++;
        }
        if (countowners == 1)
        {
            s.execute (query);
            rs = s.getResultSet ();
            rs.next ();
            actualOwner = rs.getString ("OwnerID");
            System.out.println (actualOwner);
        }
        else if (countowners > 1)
        {
            s.execute (query);
            rs = s.getResultSet ();
            String owners = "";
            while (rs.next ())
            {
                owners = owners + rs.getString ("OwnerID") + ": " +
                                rs.getString ("LastName") + ", " +
                                rs.getString ("firstName") + "\n";
            }

            actualOwner = JOptionPane.showInputDialog ("Possible
                owners below. Please enter ID Number:\n" +
                owners);
        }
        if (countowners > 0)
        {
            query = "INSERT INTO tblPets (petName, petType,
                PetBreed, OwnerID) VALUES ('" + name + "',
                '" + type + "', '" + breed + "', " +
                actualOwner + ")";
            s.execute (query);
            System.out.println ("Done.");
        }
        else
        {
            Sytem.out.println ("Owner not found");
        }
    }
}

```

```
        catch (SQLException e)
        {
            System.out.println ("ERROR: " + e);
            e.printStackTrace ();
        }
    }
}
while (choice != 'X');
}

public static void main (String[] args)
{
    new FindAPet ();
}

public char getMenuChoice ()
{
    String menu = "A) Display all owners, sorted by name\nB) Display
                  residents of a particular city\nC) Display
                  pet type statistics\nD) Show pet type
                  owners\nE) Add a new pet\nX) Exit";

    char choice = JOptionPane.showInputDialog (menu).toUpperCase ().charAt
                              (0);

    return choice;
}
}
```

Model Answer: Question 2 (JAVA)

File: Delegate.java

```
public class Delegate
{
    protected String firstName;
    protected String lastName;
    protected String company;
    protected String seminarCode;

    public Delegate (String fn, String ln, String co, String sc)
    {
        firstName = fn;
        lastName = ln;
        company = co;
        seminarCode = sc;
    }

    // Question 2.1.1
    public String toString ()
    {
        return lastName + ", " + firstName.charAt (0) + ". / " + company;
    }
    // End 2.1.1

    // Question 2.1.2
    public String getCompany ()
    {
        return company;
    }
    // End 2.1.2

    // Question 2.4.5
    public boolean attendsSeminar (char s)
    {
        if (seminarCode.indexOf (s) >= 0)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    // End 2.4.5
}
```

File: GoldDelegate.java

```
// Question 2.2
public class GoldDelegate extends Delegate
{
    // Question 2.2.1
    protected int roomNumber;

    // Question 2.2.2
    public GoldDelegate (String fn, String ln, String co, String sc, int rn)
    {
```

```

        super (fn, ln, co, sc);
        /* firstName = fn; lastName = ln; company = co; streamCode = sc; would
           also work */
        roomNumber = rn;
    }
    // End 2.2.2

    // Question 2.2.3
    public int getRoomNumber ()
    {
        return roomNumber;
    }
    // End 2.2.3

    // Question 2.2.4
    public String toString ()
    {
        return super.toString () + " / Room " + roomNumber;
    }
    // End 2.2.4
}

```

File: ConferenceManager.java

```

import java.io.*;

// Question 2.3
public class ConventionManager
{
    // Question 2.3.1
    private Delegate[] delArr = new Delegate [100];
    private int delCounter = 0;
    // End 2.3.1

    // Question 2.3.2
    public ConventionManager ()
    {
        try
        {
            BufferedReader br = new BufferedReader (new FileReader
                ("conference.txt"));

            String line = br.readLine ();
            while (line != null)
            {
                String[] temp = line.split ("#");
                if (temp.length == 4) // Delegate
                {
                    delArr [delCounter] = new Delegate (temp [0], temp [1],
                        temp [2], temp [3]);
                    delCounter++;
                }
                else if (temp.length == 5) // Gold delegate
                {
                    delArr [delCounter] = new GoldDelegate (temp [0], temp [1],
                        temp [2], temp [3], Integer.parseInt (temp
                            [4]));
                    delCounter++;
                }
                line = br.readLine ();
            }
        }
        catch (IOException e)
        {
            System.err.println ("Die.");
        }
    }
    // End 2.3.2
}

```

```

// Question 2.3.3
public String getDelegateList ()
{
    String temp = "";
    for (int i = 0 ; i < delCounter; i ++)
    {
        temp = temp + delArr [i].toString () + "\n";
    }
    return temp;
}
// End 2.3.3

// Question 2.3.4
public String getGoldDelegateList ()
{
    String temp = "";
    for (int i = 0 ; i < delCounter; i ++)
    {
        if (delArr [i] instanceof GoldDelegate)
        {
            temp = temp + delArr [i].toString () + "\n";
        }
    }
    return temp;
}
// End 2.3.4

// Question 2.3.5
public void sortDelegateCompany ()
{
    for (int i = 0 ; i < delCounter - 1 ; i ++)
    {
        for (int j = 0; j < delCounter - 1 ; j ++)
        {
            if (delArr [j].getCompany ().compareToIgnoreCase (delArr [j +
                1].getCompany ()) > 0)
            {
                Delegate temp = delArr [j];
                delArr [j] = delArr [j + 1];
                delArr [j + 1] = temp;
            }
        }
    }
}
// End 2.3.5

// Question 2.4.5
public String getSeminarNumbers ()
{
    String [] seminars = new String [4];
    seminars [0] = "Programming";
    seminars [1] = "SQL Queries";
    seminars [2] = "Object Oriented Programming";
    seminars [3] = "Database Design";

    int [] semCounter = new int [4];

    String output = "";
    for (int s = 0; s < 4; s ++)
    {
        for (int d = 0; d < delCounter; d ++)
        {
            if (delArr [d].attendsSeminar (seminars [s].charAt (0)))
            {

```

```

        semCounter [s] ++;
    }
}
    output = output + seminars [s] + "\t" + semCounter [s] + "\n";
}
    return output;
}
// End 2.4.5
}

```

File: ConferenceApp.java

```

import javax.swing.*;

public class ConventionApp
{
    public static void main (String[] args)
    {
        ConventionManager cm = new ConventionManager ();

        char choice;
        do
        {
            choice = getMenuChoice ();

            if (choice == 'A')
            {
                System.out.println (cm.getDelegateList ());
            }

            else if (choice == 'B')
            {
                System.out.println (cm.getGoldDelegateList ());
            }

            else if (choice == 'C')
            {
                cm.sortDelegateCompany ();
            }

            else if (choice == 'D')
            {
                System.out.println (cm.getSeminarNumbers ());
            }

        }
        while (choice != 'X');
    }

    public static char getMenuChoice ()
    {
        String menu = "A) List all delegates\nB) List all Gold delegates\nC)
                        Sort delegates\nD) Show the number of
                        delegates attending each seminar\nX) Exit";

        char choice = JOptionPane.showInputDialog (menu).toUpperCase ().charAt
                                (0);

        return choice;
    }
}

```

Model Answer: Question 2

(Java/Netbeans)

File: Delegate.java

```
package memo2009;

public class Delegate {

    protected String firstName;
    protected String lastName;
    protected String company;
    protected String seminarCode;

    public Delegate(String fn, String ln, String co, String sc) {
        firstName = fn;
        lastName = ln;
        company = co;
        seminarCode = sc;
    }

    // Question 2.1.1
    public String toString() {
        return lastName + ", " + firstName.charAt(0) + ". / " + company;
    }
    // End 2.1.1

    // Question 2.1.2
    public String getCompany() {
        return company;
    }
    // End 2.1.2

    // Question 2.4.5
    public boolean attendsSeminar(char s) {
        if (seminarCode.indexOf(s) >= 0) {
            return true;
        } else {
            return false;
        }
    }
    // End 2.4.5
}
```

File: GoldDelegate.java

```
package memo2009;

// Question 2.2
public class GoldDelegate extends Delegate {
    // Question 2.2.1

    protected int roomNumber;

    // Question 2.2.2
    public GoldDelegate(String fn, String ln, String co, String sc, int rn) {
        super(fn, ln, co, sc);
        // firstName = fn; lastName = ln; company = co; streamCode = sc; //
        // would also work

        roomNumber = rn;
    }
}
```

```

    }
    // End 2.2.2

    // Question 2.2.3
    public int getRoomNumber ()
    {
        return roomNumber;
    }
    // End 2.2.3

    // Question 2.2.4
    public String toString() {
        return super.toString() + " / Room " + roomNumber;
    }
    // End 2.2.4
}

```

File: ConventionManager.java

```

package memo2009;

import java.io.*;

// Question 2.3
public class ConventionManager
{
    // Question 2.3.1
    private Delegate[] delArr = new Delegate [100];
    private int delCounter = 0;
    // End 2.3.1

    // Question 2.3.2
    public ConventionManager ()
    {
        try
        {
            BufferedReader br = new BufferedReader
            (new FileReader ("conference.txt"));

            String line = br.readLine ();

            while (line != null)
            {
                String[] temp = line.split("#");
                if (temp.length == 4) // Delegate
                {
                    delArr [delCounter] = new Delegate
                    (temp [0], temp [1], temp [2], temp [3]);
                    delCounter++;
                }
                else if (temp.length == 5) // Gold delegate
                {
                    delArr [delCounter] = new GoldDelegate
                    (temp [0], temp [1], temp [2], temp [3],
                    Integer.parseInt (temp [4]));
                    delCounter++;
                }
                line = br.readLine ();
            }
        }
        catch (IOException e)

```

```
        {
            System.err.println ("Die.");
        }
    }
// End 2.3.2

// Question 2.3.3
public String getDelegateList ()
{
    String temp = "";
    for (int i = 0 ; i < delCounter; i ++)
    {
        temp = temp + delArr [i].toString () +
        "\n";
    }
    return temp;
}
// End 2.3.3

// Question 2.3.4
public String getGoldDelegateList ()
{
    String temp = "";
    for (int i = 0 ; i < delCounter; i ++)
    {
        if (delArr [i] instanceof GoldDelegate)
        {
            temp = temp + delArr [i].toString () +
            "\n";
        }
    }
    return temp;
}
// End 2.3.4

// Question 2.3.5
public void sortDelegateCompany ()
{
    for (int i = 0 ; i < delCounter - 1 ; i ++)
    {
        for (int j = 0; j < delCounter - 1 ; j
        ++){
            {
                if (delArr [j].getCompany
                ().compareToIgnoreCase (delArr [j +
                1].getCompany ()) > 0)
                {
                    Delegate temp = delArr [j];
                    delArr [j] = delArr [j + 1];
                    delArr [j + 1] = temp;
                }
            }
        }
    }
}
// End 2.3.5

// Question 2.4.5
public String getSeminarNumbers ()
{
    String [] seminars = new String [4];
    seminars [0] = "Programming";
    seminars [1] = "SQL Queries";
    seminars [2] = "Object Oriented
    Programming";
    seminars [3] = "Database Design";
}
```

```

        int [] semCounter = new int [4];

        String output = "";
        for (int s = 0; s < 4; s ++)
        {
            for (int d = 0; d < delCounter; d ++)
            {
                if (delArr [d].attendsSeminar (seminars
                [s].charAt (0)))
                {
                    semCounter [s] ++;
                }
            }

            output = output + seminars [s] + "\t" +
            semCounter [s] + "\n";
        }

        return output;
    }
    // End 2.4.5
}

```

File: ConventionApp.java

```

package memo2009;

/**
 *
 * @author Delia
 */
public class ConventionApp extends javax.swing.JFrame {

    /** Creates new form ConventionApp */
    public ConventionApp() {
        initComponents();
    }
    ConventionManager cm = new ConventionManager();

    /** This method is called from within the constructor to
     * initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is
     * always regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jScrollPane1 = new javax.swing.JScrollPane();
        txaOutput = new javax.swing.JTextArea();
        jMenuBar1 = new javax.swing.JMenuBar();
        mnuMain = new javax.swing.JMenu();
        mnuDisplayAllDelegates = new javax.swing.JMenuItem();
        mnuDisplayGoldDelegates = new javax.swing.JMenuItem();
        mnuSortDelegates = new javax.swing.JMenuItem();
        mnuPeopleAttendingSeminars = new javax.swing.JMenuItem();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("IEB IT Prac 2009");

        txaOutput.setColumns(20);
        txaOutput.setRows(5);
        jScrollPane1.setViewportView(txaOutput);
    }
}

```

```

mnuMain.setText("Menu");

mnuDisplayAllDelegates.setText("Display all Delegates");
mnuDisplayAllDelegates.addActionListener(new
    java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuDisplayAllDelegatesActionPerformed(evt);
        }
    });
mnuMain.add(mnuDisplayAllDelegates);

mnuDisplayGoldDelegates.setText("Display Gold Delegates");
mnuDisplayGoldDelegates.addActionListener(new
    java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuDisplayGoldDelegatesActionPerformed(evt);
        }
    });
mnuMain.add(mnuDisplayGoldDelegates);

mnuSortDelegates.setText("Sort Delegates");
mnuSortDelegates.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        mnuSortDelegatesActionPerformed(evt);
    }
});
mnuMain.add(mnuSortDelegates);

mnuPeopleAttendingSeminars.setText("People Attending Seminars");
mnuPeopleAttendingSeminars.addActionListener(new
    java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            mnuPeopleAttendingSeminarsActionPerformed(evt);
        }
    });
mnuMain.add(mnuPeopleAttendingSeminars);

jMenuBar1.add(mnuMain);

setJMenuBar(jMenuBar1);

javax.swing.GroupLayout layout = new
    javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(10, 10, 10)
        .addComponent(jScrollPane1,
            javax.swing.GroupLayout.DEFAULT_SIZE, 380,
            Short.MAX_VALUE)
        .addGap(10, 10, 10)
    );
layout.setVerticalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGroup(layout.createSequentialGroup()
        .addGap(10, 10, 10)
        .addComponent(jScrollPane1,
            javax.swing.GroupLayout.DEFAULT_SIZE, 257,
            Short.MAX_VALUE)

```

```

        .addContainerGap()
    );

    pack();
} // </editor-fold>

private void
                                mnuDisplayAllDelegatesActionPerformed(java.a
                                wt.event.ActionEvent evt) {
    txaOutput.setText(cm.getDelegateList());
}

private void
                                mnuDisplayGoldDelegatesActionPerformed(java.
                                awt.event.ActionEvent evt) {
    txaOutput.setText(cm.getGoldDelegateList());
}

private void mnuSortDelegatesActionPerformed(java.awt.event.ActionEvent
                                evt) {
    cm.sortDelegateCompany();
    txaOutput.setText(cm.getDelegateList());
}

private void
                                mnuPeopleAttendingSeminarsActionPerformed(ja
                                va.awt.event.ActionEvent evt) {
    txaOutput.setText(cm.getSeminarNumbers());
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    java.awt.EventQueue.invokeLater(new Runnable() {

        public void run() {
            new ConventionApp().setVisible(true);
        }
    });
}

// Variables declaration - do not modify
private javax.swing.JMenuBar jMenuBar1;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JMenuItem mnuDisplayAllDelegates;
private javax.swing.JMenuItem mnuDisplayGoldDelegates;
private javax.swing.JMenu mnuMain;
private javax.swing.JMenuItem mnuPeopleAttendingSeminars;
private javax.swing.JMenuItem mnuSortDelegates;
private javax.swing.JTextArea txaOutput;
// End of variables declaration
}

```

Model Answer: Question 1 (Delphi)

File: Question1.pas

```

unit Question1;

interface

uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
  Dialogs, DB, ADODB, Menus, StdCtrls, ComCtrls, Grids, DBGrids;

type
  TForm1 = class(TForm)
    qry1: TADOQuery;
    mm1: TMainMenu;
    Queries1: TMenuItem;
    OwnersSortedbyLastnameFirstname1: TMenuItem;
    Allownersforacity1: TMenuItem;
    Countpettypes1: TMenuItem;
    Ownersforaspecificpettype1: TMenuItem;
    Addapet1: TMenuItem;
    ds1: TDataSource;
    dbgrd1: TDBGrid;
    procedure Allownersforacity1Click(Sender: TObject);
    procedure Addapet1Click(Sender: TObject);
    procedure Countpettypes1Click(Sender: TObject);
    procedure Ownersforaspecificpettype1Click(Sender: TObject);
    procedure OwnersSortedbyLastnameFirstname1Click(Sender: TObject);
    procedure QryDB (s : string);
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

procedure TForm1.Addapet1Click(Sender: TObject);
var
  Owners: string;
  ownerid: string;
  petbreed: string;
  ownername: string;
  pettype: string;
  petname: string;
begin
  // add a pet
  petname := InputBox('New Pet Details', 'Please enter the pet''s name', '');
  pettype := InputBox('New Pet Details', 'Please enter the pet''s type', '');
  petbreed := InputBox('New Pet Details', 'Please enter the pet''s breed', '');
  ownername := InputBox('New Pet Details', 'Please enter the owner''s name',
    '');
  QryDB('SELECT * FROM tblOwners WHERE lastname = '' + ownername + ''');
  qry1.Open;
  if qry1.RecordCount = 1 then
    begin
      ownerid := qry1.fieldbyname ('ownerid').asstring;
    end
  else if qry1.RecordCount > 1 then

```

```

begin
    qry1.First;
    while not qry1.Eof do
        begin
            Owners := Owners + qry1.FieldName('ownerid').AsString + ' : ' +
                        qry1.FieldName('lastname').AsString + ', ' +
                        qry1.FieldName('firstname').AsString + #13;
            qry1.Next;
        end;
        Ownerid := InputBox('Choose the owner', 'Enter the ID from the list ' +
                            'below'+#13+owners, '');
    end
    else
        begin
            MessageDlg ('Owner not found', mtError, [mbok],0);
        end;
    end;

    QryDB('INSERT INTO tblPets (petName, petType, PetBreed, OwnerID) VALUES (''
        + petname + ''', '' + pettype + ''', '' + petbreed + ''', ''
        + Ownerid + ''')');
end;

procedure TForm1.Allowersforacity1Click(Sender: TObject);
var
    cityname: string;
begin
    // list all owners for a city
    cityname := InputBox('Select a city:', 'Type in the city name:', '');
    QryDB('SELECT * FROM tblOwners WHERE city = '' + cityname +
        '' ORDER BY LastName, FirstName');
    qry1.Open;
end;

procedure TForm1.Countpettypes1Click(Sender: TObject);
begin
    // count pet types
    QryDB('SELECT PetType, COUNT(*) as num FROM tblPets GROUP BY PetType');
    qry1.Open;
end;

procedure TForm1.Ownersforaspecificpettype1Click(Sender: TObject);
var
    pettype: string;
begin
    // list all owners for a pet type
    pettype := InputBox('Pet type selector', 'Type in the pet type', '');
    QryDB('SELECT DISTINCT tblOwners.* FROM tblOwners INNER JOIN tblPets ON ' +
        'tblOwners.ownerID = tblPets.OwnerID WHERE PetType = '' +
        pettype + '' ORDER BY lastname, firstname');
    qry1.Open;
end;

procedure TForm1.OwnersSortedbyLastnameFirstname1Click(Sender: TObject);
begin
    // list all owners sorted
    QryDB('SELECT * FROM tblOwners ORDER BY LastName, FirstName');
    qry1.Open;
end;

procedure TForm1.QryDB(s: string);
begin
    qry1.Active := false;
    qry1.SQL.Text := s;
    qry1.ExecSQL;
end;

end.

```

Model Answer: Question 2 (Delphi)

File: uDelegate.pas

```
unit uDelegate;

interface

uses SysUtils, Math, Dialogs;

type Delegate = class(tobject)
    private

    protected
        firstname : string;
        lastname : string;
        company : string;
        streamcodes : string;
    public
        constructor create (fn, ln, c, sc : string);
        Function ToString : string; virtual;
        function GetCompany : string;
        function CheckSession (ss : string) : Boolean;
    end;

implementation

{ Delegate }
function Delegate.CheckSession(ss: string): Boolean;
begin
    Result := False;
    if Pos(uppercase (ss), uppercase (streamcodes)) <> 0 then
        Result := True;
end;

constructor Delegate.create(fn, ln, c, sc: string);
begin
    firstname := fn;
    lastname := ln;
    company := c;
    streamcodes := sc;
end;

function Delegate.GetCompany: string;
begin
    result := company;
end;

function Delegate.ToString: string;
begin
    Result := lastname + ', ' + firstname[1] + ' / ' + company;
end;

end.
```

File: uGoldDelegate.pas

```
unit uGoldDelegate;

interface
```

```

uses uDelegate;

Type GoldDelegate = class (Delegate)
    private
        RoomNo : string;
    protected
    public
        constructor Create (fn, ln, c, sc, rn: string);
        function GetRoomNo : string;
        function ToString : string; override;
    end;

implementation

{ GoldDelegate }

constructor GoldDelegate.Create(fn, ln, c, sc, rn: string);
begin
    inherited Create(fn, ln, c, sc);
    RoomNo := rn;
end;

function GoldDelegate.GetRoomNo: string;
begin
    Result := RoomNo;
end;

function GoldDelegate.ToString: string;
begin
    Result := inherited ToString + ' / Room ' + RoomNo;
end;

end.

```

File: uConferenceManager.pas

```

unit uConventionManager;

interface

uses uDelegate, uGoldDelegate, SysUtils, Dialogs, math;

type ConventionManager = class (TObject)
    private
        ArrDelegates : array [1..100] of Delegate;
        NoOfDelegates : Integer;
    protected
    public
        constructor Create;
        function GetDelegateList : string;
        function GetGoldelegateList : string;
        procedure SortDelegateCompany;
        function AttendanceFigures : string;
    end;

implementation

{ ConventionManager }

function ConventionManager.AttendanceFigures: string;
var
    sCnt: integer;
    DCnt: integer;
    PCnt: integer;

```

```

    OCnt: integer;
    index: Integer;
begin
    OCnt := 0;
    PCnt := 0;
    DCnt := 0;
    sCnt := 0;
    for index := 1 to NoOfDelegates do
        begin
            if ArrDelegates [index].checksession ('o') then Inc(OCnt);
            if ArrDelegates [index].checksession ('p') then Inc(pCnt);
            if ArrDelegates [index].checksession ('d') then Inc(dCnt);
            if ArrDelegates [index].checksession ('s') then Inc(sCnt);
        end;
    Result := 'The delegate counts per session are: ' + #13 +
        'Programming:' + #9 + IntToStr(pcnt) + #13 +
        'SQL:' + #9 + IntToStr(scnt) + #13 +
        'OOP:' + #9 + IntToStr(ocnt) + #13 +
        'Database:' + #9 + IntToStr(dcnt);
end;

constructor ConventionManager.Create;
var
    roomno: string;
    sc: string;
    Cmpny: string;
    Ln: string;
    fn: string;
    tmp: string;
    df: Textfile;
begin
    NoOfDelegates := 0;
    AssignFile(df, 'conference.txt');
    Reset(df);
    while not Eof(df) do
        begin
            Readln(df, tmp);
            fn := Copy(tmp, 1, Pos ('#', tmp) -1);
            Delete (tmp, 1, Pos ('#', tmp));
            Ln := Copy(tmp, 1, Pos ('#', tmp) -1);
            Delete (tmp, 1, Pos ('#', tmp));
            Cmpny := Copy(tmp, 1, Pos ('#', tmp) -1);
            Delete (tmp, 1, Pos ('#', tmp));
            if Pos ('#', tmp) = 0 then
                begin
                    sc := tmp;
                    roomno := '';
                end
            else
                begin
                    sc := Copy(tmp, 1, Pos ('#', tmp) -1);
                    Delete (tmp, 1, Pos ('#', tmp));
                    roomno := tmp;
                end;
            Inc(noOfDelegates);
            if roomno = '' then
                ArrDelegates [NoOfDelegates] := Delegate.create(fn, Ln, Cmpny, sc)
            else
                ArrDelegates [NoOfDelegates] := GoldDelegate.create(fn, Ln, Cmpny, sc,
                    roomno);
        end;
    end;
end;

function ConventionManager.GetDelegateList: string;
var

```

```

    tmp: string;
    index: integer;
begin
    tmp := '';
    for index := 1 to NoOfDelegates do
        tmp := tmp + ArrDelegates[NoOfDelegates].ToString;
    Result := tmp;
end;

function ConventionManager.GetGoldelegateList: string;
var
    tmp: string;
    index: integer;
begin
    tmp := '';
    for index := 1 to NoOfDelegates do
        if ArrDelegates[NoOfDelegates] is GoldDelegate then
            tmp := tmp + ArrDelegates[NoOfDelegates].ToString;
    Result := tmp;
end;

procedure ConventionManager.SortDelegateCompany;
var
    tmp: Delegate;
    Index2: Integer;
    index: Integer;
begin
    for index := 1 to NoOfDelegates - 1 do
        begin
            for Index2 := index + 1 to NoOfDelegates do
                begin
                    if ArrDelegates[index].GetCompany > ArrDelegates[Index2].GetCompany
                        then
                            begin
                                tmp := ArrDelegates[index];
                                ArrDelegates[index] := ArrDelegates[Index2];
                                ArrDelegates[Index2] := tmp;
                            end;
                end;
            end;
        end;
    end;
end.

```

File: ufrmMain.pas

```

unit ufrmMain;

interface

uses
    Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
    Dialogs, Menus, StdCtrls, ComCtrls;

type
    TForm1 = class(TForm)
        mm1: TMainMenu;
        Options1: TMenuItem;
        Quit1: TMenuItem;
        DisplayAllDelegates1: TMenuItem;
        DisplayGoldDelegates1: TMenuItem;
        SortDelegates1: TMenuItem;
        Displayattendancefigures1: TMenuItem;
        redtDisplay: TRichEdit;
    end;

```

```
    procedure Quit1Click(Sender: TObject);
    procedure DisplayAllDelegates1Click(Sender: TObject);
    procedure FormActivate(Sender: TObject);
    procedure DisplayGoldDelegates1Click(Sender: TObject);
    procedure SortDelegates1Click(Sender: TObject);
    procedure Displayattendancefigures1Click(Sender: TObject);
private
    { Private declarations }
public
    { Public declarations }
end;

var
    Form1: TForm1;

implementation

uses uConventionManager;

var MyConvention : ConventionManager;

{$R *.dfm}

procedure TForm1.DisplayAllDelegates1Click(Sender: TObject);
begin
    redtDisplay.Lines.Clear;
    redtDisplay.Lines.Add(MyConvention.GetDelegateList);
end;

procedure TForm1.Displayattendancefigures1Click(Sender: TObject);
begin
    redtDisplay.Lines.Clear;
    redtDisplay.Lines.Add(MyConvention.GetGoldelegateList);
end;

procedure TForm1.DisplayGoldDelegates1Click(Sender: TObject);
begin
    redtDisplay.Lines.Clear;
    redtDisplay.Lines.Add(MyConvention.GetGoldelegateList);
end;

procedure TForm1.FormActivate(Sender: TObject);
begin
    MyConvention := ConventionManager.Create;
end;

procedure TForm1.Quit1Click(Sender: TObject);
begin
    Close;
end;

procedure TForm1.SortDelegates1Click(Sender: TObject);
begin
    MyConvention.SortDelegateCompany;
end;

end.
```

Centre Number:	
Candidate Number:	
Marker's Name:	

TOTAL: 0 0%

Q	Description	Mark	Pupil
1.1	Query: correct fields (or *), ✓ correct table, ✓ correct ORDER BY fields, ✓✓ correct output is displayed ✓ SELECT * FROM tblOwners ORDER BY lastName, firstName	5	
1.2	Input city, ✓ build input into query, ✓ correct fields (or *), correct table, ✓ correct WHERE, ✓ correct output is displayed ✓ SELECT * FROM tblOwners WHERE city =''' + city + ''';	5	
1.3	Field includes COUNT(*) ✓ with an alias, ✓ petType ✓ uses the correct table, ✓ GROUP BY ✓ petType ✓ Correct output produced ✓	7	
1.4	Input pet type, ✓ build input into query, ✓ correct fields in SELECT, ✓ uses tables tblOwners and tblPets, ✓ an INNER JOIN is used, ✓ correct JOIN condition, ✓ correct WHERE , ✓ correct output is produced. ✓ NB: If the pupil has used a WHERE clause instead of an INNER JOIN, transfer the marks for the correct condition ✓ and the correct Boolean amendment to the WHERE (using AND) ✓	8	
1.5	Input pet name, ✓ breed, ✓ type, ✓ input owner name, ✓ search for owner name using SELECT query, ✓✓ check if more than one owner exists, ✓ either choose single owner or ask for user input, ✓✓ build fields into an INSERT query, ✓✓✓✓ INSERT has correct syntax, ✓query is executed correctly. ✓	15	
2.1.1	Correct fields, ✓ correct string handling for initial, ✓ method correctly named ✓ and typed ✓ returns modified string ✓	5	
2.1.2	Method is named get ... , ✓ method is appropriately typed, ✓ returns correct value. ✓	3	
2.2	Class is named correctly, ✓ class inherits from delegate ✓	2	
2.2.1	Instance variable is an integer, ✓ variable has a descriptive name, ✓ scope is set to private (or protected). ✓	3	
2.2.2	Method is named correctly, ✓ parameters are correct, ✓ initialises parent instance variables ✓ initialises local instance variables. ✓	4	
2.2.3	Correct field ✓ is returned, ✓ method is appropriately named get ... ✓ and appropriately typed ✓	4	
2.2.4	Method calls toString method from parent class or re-builds parent string from variables, ✓ Room number is concatenated with 'Room' text, ✓ returns String ✓	3	

Centre/ Candidate:			
2.3	New class is declared correctly ✓ and named correctly ✓	2	
2.3.1	Array is declared correctly ✓✓ to store 100 objects, ✓ an integer is declared to count the contents ✓✓	5	
2.3.2	File is opened successfully for reading, ✓✓ indefinite loop to read in lines ✓ with correct termination condition, ✓ reads new lines from file, ✓ correctly determines whether line is a Delegate or GoldDelegate (any working code), ✓✓ creates new Delegate or GoldDelegate object, ✓ correct parameters for Delegate ✓ and correct parameters for GoldDelegate, ✓ stores the object in the array, ✓ increments array counter, ✓ includes some working IO Exception handling/ throwing ✓	13	
2.3.3	Use a for loop, ✓ with the correct starting and termination points, ✓ output is appended to a temporary variable, ✓ the toString method is used, ✓ delegates are on individual lines, ✓ temp variable is returned ✓	6	
2.3.4	Uses same looping technique as above ✓ (if not, award one mark for functioning loop), if-statement inside loop, ✓ correct test for instance of GoldDelegate, ✓ toString method is used. ✓	4	
2.3.5	Any functioning sort from first principles that compares the companies based on the getCompany method: 8 marks. Otherwise: outer loop with correct bounds, ✓✓ inner loop with correct bounds, ✓✓ check to see if elements need to swap, ✓✓ swapping elements ✓✓	8	
2.4.1	correct place, ✓ correct instantiation ✓	2	
2.4.2	call appropriate method, ✓ display output ✓	2	
2.4.3	call appropriate method, ✓ display output ✓	2	
2.4.4	call appropriate method ✓	1	
2.4.5	Working solution: Level 3: A working solution is employed that takes into account the data provided (4 to 5 marks) Level 2: A working solution is provided but which either suffers from logical errors or false assumptions about the code (e.g. that the codes are all in the same order) (2 to 3 marks) Level 1: A working solution that is not based on the data provided in the object or file (0 to 1 marks). Deduct marks for syntax and logical errors (max of 1 each).	5	
	OOP Principles: Level 3: Ideally, code exists at ConventionManager and Delegate level to isolate the responsibility of the parts of the solution, parameter passing and value returning is well-employed (5 to 6 marks). Level 2: The solution makes use of parameter passing and value returning, but issues arise about the location of the code used to solve the problem, e.g. mostly done in App program (3 to 4 marks). Level 1: The solution is poorly constructed and haphazard. A single method/ block of code attempts to solve the problem, principles of encapsulation are ignored (1 to 2 marks).	6	